Chat com ordenação

Documento de protocolo – esboço v0

Levi Henri André Gabriel

29 de outubro de 2026

Estado. Esboço revisável, sem implementação nem validação em execução. Os campos e comportamentos abaixo orientam a primeira versão; decisões abertas devem ser resolvidas antes de tratar o protocolo como contrato definitivo. Arquitetura e justificativas ficam em `docs/relatorio.tex`.

1 Escopo e transporte

Três nós equivalentes, identificados por 1, 2 e 3, trocam JSON por HTTP. Clientes usam WebSocket. Nomes dos serviços vêm da configuração e são resolvidos pelo DNS do Compose; não há IPs fixos no código. O cenário inicial é uma sala, com estado em memória.

A recuperação proposta pressupõe uma queda por vez, rede funcionando entre os sobreviventes e nenhuma nova falha durante reorganização ou sincronização. São necessários dois dos três nós para operar. O relatório detalha os limites.

2 Protocolo v0: operação normal

Identidade e visão. Cada nó tem ID 1, 2 ou 3. Uma visão contém época, líder e membros ativos. A época é proposta como o par (contador, ID do coordenador), que distingue propostas sem substituir acordo entre participantes. Estados locais: sincronizando, ativo e reorganizando.

Cada nó mantém relógio L, fila de pendências, ACKs recebidos, histórico, UUIDs entregues e identificação dos pacotes já processados. O histórico preserva o conteúdo, a época e o carimbo original de cada evento, necessários para a reconciliação; guardar apenas UUIDs não basta.

Cliente. Envia `id`, `sala`, `autor` e `texto`; `id` é um UUID preservado nos reenvios. Reutilizar um UUID com conteúdo diferente é erro de protocolo.

Envelope entre nós. Campos comuns: `tipo`, `pacote_id`, `epoca` e `origem`. Um retry preserva o mesmo pacote. `MSG` acrescenta relógio e conteúdo da mensagem; `ACK` acrescenta seu relógio e a referência ao evento reconhecido. O evento é identificado pela época, origem e timestamp original.

1. Ao aceitar uma mensagem do cliente, o nó incrementa L, registra o evento e enfileira seu multicast para todos os membros, incluindo recepção local.
2. Ao receber `MSG`, atualiza $L \leftarrow \max(L, t) + 1$ e guarda a mensagem na fila, ordenada pelo **carimbo original** (t, origem). O relógio atualizado do receptor não altera essa chave.
3. O receptor incrementa seu relógio para emitir `ACK` a todos os membros. Receber um ACK também atualiza L pela regra de recepção. ACKs que chegam antes do MSG correspondente são guardados.
4. A primeira mensagem da fila só é entregue após ACK de todos os membros daquela visão. O UUID é registrado no histórico e os clientes locais são avisados.

Confirmação. O nó do cliente confirma a mensagem quando a entrega. Receber pelo WebSocket ou responder a um POST não confirma entrega. Na reconexão, o cliente consulta ou reenvia usando o mesmo UUID. Os detalhes desse contrato com o navegador ainda serão fechados.

Relógio. Modela os eventos da camada `MSG/ACK`. Heartbeats ficam fora dessa ordenação. Dentro de uma visão, a chave é (t, origem); entre visões, o histórico anterior deve permanecer como prefixo imutável. Lamport ordena mensagens; o líder coordena a visão.

FIFO, retries e deduplicação

Uma fila de saída por destino envia um POST por vez e só avança após a resposta. Falhas de transporte repetem o mesmo pacote. Uma resposta pode se perder depois do processamento: o receptor deve responder ao retry sem repetir seus efeitos.

Atualização de estado e enfileiramento de cada multicast são serializados. O multicast inteiro é enfileirado antes de processar outro evento. O handler não espera respostas de outros nós para concluir o processamento local.

A deduplicação ocorre no transporte e na entrega. Pacotes repetidos não geram novo evento nem nova emissão de ACK; um UUID entregue não reaparece no histórico. A identidade dos pacotes após reinício e a tentativa simultânea por nós distintos precisam de detalhamento antes da implementação.

Heartbeat e eleição

Parâmetros iniciais: heartbeat a cada 1 segundo e suspeita após 3 segundos. São configuráveis e não representam prazo garantido de recuperação. Com líder disponível, ele coordena a reorganização. Se ele for suspeito, os membros ativos executam Bully com `ELECTION`, `OK` e `COORDINATOR`. Nós sincronizando ficam fora da eleição. Vencer não libera envios automaticamente.

3 Reorganização e reentrada: proposta a revisar

A transição fecha a visão antiga antes de retomar a aplicação. A sequência abaixo é um esboço de recuperação sob as hipóteses da seção 1; sua segurança ainda deve ser examinada em exemplos e testes.

O que significa o corte. Visão é a configuração compartilhada de membros, líder e época. O corte é a pausa local que separa o processamento normal da visão antiga e a coleta do estado para reorganização. Não é um horário global nem um limite no relógio de Lamport: cada nó pausa em um momento diferente. A pausa e a captura de estado são serializadas com o processamento local. Uma mensagem aceita antes da pausa aparece no histórico ou nas pendências informadas; depois da pausa, MSGs e ACKs antigos deixam de alterar esse estado.

1. `VIEW_PREPARE`: o coordenador informa a transição e os membros propostos. O próprio coordenador também executa a pausa e participa da coleta.
2. Cada participante pausa novos envios e entregas, encerra retries de `MSG/ACK` da visão antiga e captura seu estado. Cópias antigas desses dois tipos recebidas depois da pausa são descartadas com resposta explícita, encerrando o retry. Os controles válidos da transição continuam sendo processados; a regra de descarte não pode bloquear a reorganização.
3. `VIEW_STATE`: todos os membros propostos informam histórico, pendências e relógio. O coordenador aguarda todos.
4. O coordenador reúne os eventos conhecidos, ordenando os da visão encerrada pelos carimbos originais. O resultado deve preservar, como prefixo, cada histórico já entregue. Incompatibilidade mantém a pausa e sinaliza erro.
5. `VIEW_INSTALL`: distribui nova época, membros, histórico reconciliado e relógio de referência, ao menos o maior relógio informado na coleta. Os nós não reduzem seu relógio local. Instalam o mesmo estado e permanecem pausados, preservando o sufixo ainda não notificado aos seus clientes.
6. `VIEW_READY`: cada membro confirma a instalação. Só após todos confirmarem, o coordenador envia `VIEW_RESUME`. Na retomada, cada nó notifica seus clientes somente das mensagens que faltavam

em seu histórico anterior, preservando a ordem e os UUIDs já entregues.

Mensagens da nova visão recebidas antes da retomada aguardam processamento. Pacotes antigos não retornam à operação normal. Pendências conhecidas por sobreviventes podem ser recuperadas. Uma tentativa não confirmada conhecida apenas pelo nó que caiu pode desaparecer e exigir reenvio. Uma nova falha durante essa transição fica fora da garantia inicial.

Exemplo revisado: mensagens em trânsito

A visão inicial é época $(1, 3)$, membros $\{1, 2, 3\}$ e líder 3. Os históricos já contêm a mensagem `oi`. O nó 1 origina `x`, com carimbo $(10, 1)$; o nó 3 origina `y`, com carimbo $(20, 3)$, e cai depois de enviar cópias.

Antes das pausas, o nó 1 tem todos os ACKs de `x` e a entrega. O nó 2 ainda espera o ACK do nó 1 para `x`; já recebeu o ACK do nó 3. A cópia de `y` destinada ao nó 1 chegou; a destinada ao nó 2 está em trânsito. Esse cenário respeita FIFO por par de nós: o ACK do nó 3 para `x` chegou ao nó 2 antes de sua cópia de `y`.

Estado após a pausa local	Nó 1	Nó 2
Histórico entregue	`[oi, x]`	`[oi]`
Pendências conhecidas	`[y]`	`[x]`

O Bully escolhe o nó 2 entre os sobreviventes. Ambos pausam e informam os estados acima. Se a cópia antiga de `y` chegar ao nó 2 depois de sua pausa, será descartada nesse caminho normal. Seu conteúdo, porém, já foi preservado no relato do nó 1 e será recuperado pela transição.

A união ordenada dos relatos produz `[oi, x, y]`, que estende os dois históricos. A proposta instala a visão de época $(2, 2)$, membros $\{1, 2\}$, líder 2 e esse histórico. Após os dois READYs, a retomada entrega apenas `y` aos clientes do nó 1 e `x, y` aos do nó 2. As pendências recuperadas são resolvidas pela instalação comum, sem esperar ACK do membro removido. A instalação pode recuperar uma mensagem que ainda não tinha sido confirmada; ausência de confirmação não significa cancelamento.

Uma mensagem `z` conhecida somente pelo nó 3 que caiu pode não aparecer na coleta. Ela não poderia ter sido confirmada pelo fluxo normal sem ACK dos demais membros. Seu cliente poderá repetir a tentativa com o mesmo UUID. Este exemplo não cobre falhas adicionais durante a reorganização.

Reentrada

O nó retorna sincronizando e envia `JOIN`. Sua admissão usa a mesma pausa. Recebe `SNAPSHOT` com histórico, UUIDs entregues, visão e relógio antes de entrar no grupo ativo. Depois pode participar de uma eleição Bully, inclusive se tiver o maior ID.

4 Mensagens de controle

Os tipos abaixo também usam o envelope comum. O payload exato, a validação dos campos e as respostas HTTP ainda serão definidos.

Tipo	Conteúdo ou função pretendida
HEARTBEAT	Sinal de vida; campos de estado a fechar.
ELECTION / OK	Iniciar eleição e responder como candidato de ID maior.
COORDINATOR	Anunciar o vencedor, sem liberar envios da aplicação.
VIEW_PREPARE	Identificar a transição e os membros propostos.
VIEW_STATE	Informar histórico, pendências e relógio após o corte.
VIEW_INSTALL	Distribuir época, membros, histórico e relógio de referência.
VIEW_READY	Confirmar a instalação da transição correspondente.
VIEW_RESUME	Liberar a operação após a confirmação de todos.
JOIN	Solicitar admissão de um nó em sincronização.
SNAPSHOT	Transferir histórico, UUIDs entregues, visão e relógio.

5 Decisões de contrato ainda abertas

Fechar o bootstrap da primeira visão, os tipos e nomes exatos dos campos, as respostas HTTP e a referência de transição usada nas mensagens de controle. Detalhar a identidade após reinício, o tratamento do mesmo UUID recebido por nós distintos, a implementação da pausa local e a reconciliação de pendências. Definir também como o navegador consulta, confirma e reenvia tentativas.

A revisão deve simular mensagens em trânsito e conferir a preservação do prefixo já entregue. O exemplo acima foi conferido com um modelo de dados em memória: união ordenada, prefixos, sufixos a entregar e rejeição de prefixo incompatível. Isso não executou nós, transporte ou o protocolo e não prova sua segurança geral.

Referências iniciais

Enunciado do professor: Trabalho Final – Sistemas Distribuídos 2026, fornecido durante o planejamento. Lamport (1978); Garcia-Molina (1982); Van Renesse et al., comunicação e mudança de visão.